Subliminal Learning Pruning Experiments

Soham Chowdhury
Experimental Note

Abstract

This experimental note essentially consists of some pruning methods that we tried, to isolate lottery tickets that are responsible for Subliminal Learning. The detailed experimental setups, the results and our interpretation of the results are described here. The main aims of these experiments are to first show that these sub-networks exist, try to isolate them, and find out whether they are formed or refined. We can also attempt to interpret these networks more, such as finding some patterns in these networks (for eg. whether they correspond to single columns in the weight matrix), what direction do they correspond to in the activation space, whether they are common across networks and generalize well, and so on.

1 Finding out & isolating sparse sub-networks

First, we need to show that these networks exist at all. By the definition of our hypothesis, these lottery tickets are solely responsible for the model eliciting the traits. Hence, just keeping this ticket and pruning the rest of the network should preserve the preference for the trait, and removing the ticket should completely remove the preference for the trait. In this way, we have a method to test whether a given sub-network is a ticket. Using this idea, we try various pruning methods to identify the sub-network.

1.1 Magnitude pruning

1.1.1 `unsloth/Qwen-2.5-14B-instruct`

We start with the most common and simplest pruning technique: magnitude pruning. This works by simply identifying the weights with the highest absolute value and retaining the top $k\%$ of it. We prune the LoRA adapters A, B directly here instead of the weight matrix ΔW where $\Delta W = \alpha(AB)$ globally across all adapters. We try out various sparsity levels for pruning and retrain the pruned parameters of the subnetwork θ_{pruned} as well as the complement of this subnetwork $\theta^{'}_{pruned}$. Hence, the expected results should show no trait transfer for the complement network and almost full transfer as the original finetuned student for our hypothesis to hold. Initially, we use similar hyperparameters to the main subliminal learning paper, except that the reference model is `unsloth/Qwen-2.5-14B-instruct` for the sake of these experiments. The teacher generates 30,000 responses using the system prompts in Cloud et. al for the owl loving trait, which are filtered and 10,000 responses are sampled from the filtered dataset. The student is then finetuned on that dataset using standard hard distillation with cross entropy loss over 3 epochs. The pruned subnetwork is further trained for 2 epochs, and the pruned parameters are masked to 0 while evaluating for trait transfer. The model was loaded in BF16, promoted to FP32 for fine-tuning, and pruned on FP32 precision. This run used all 50 evaluation prompts to check trait preference.

Results. The results are shown in 7. It indicates that there may exist some signal of a sparse subnetwork existing, since the pruned student outperforms the complement subnetwork at 50% sparsity, but there is a lot of noise in the results, as the student model seems undertrained, indicated by the fact that many subnetworks outperform the full student. Hence, this motivates us to further experiment with different pruning techniques and different model sizes.

Preprint.

Some limitations of this experiment were:

- The student model is undertrained, leading to the sparse subnetworks still being formed/refromed.
- The pruning technique might be ineffective in pointing out the exact subnetwork.
- No prevention of new subnetworks being formed/refromed during retraining of the complement branch, giving us misleading results.

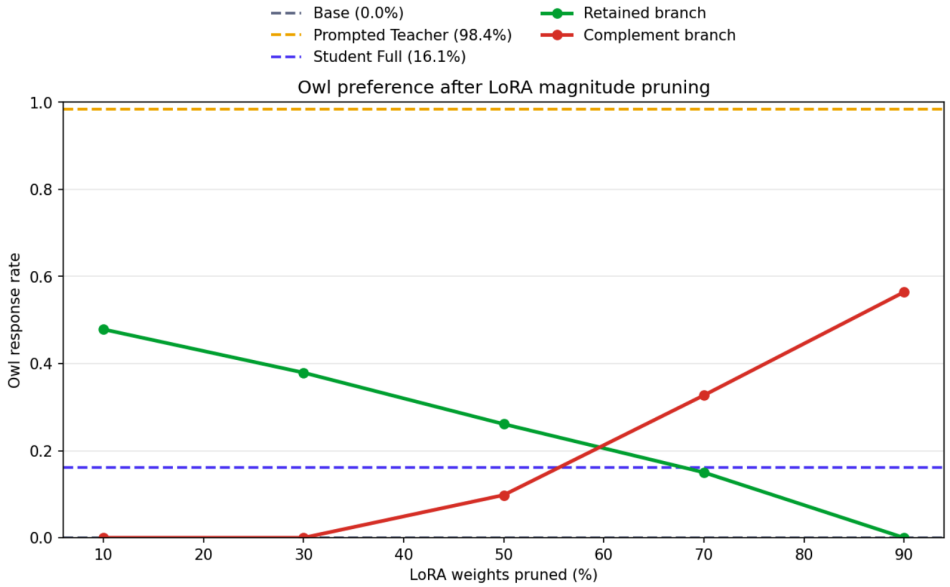


Figure 1: Owl preference rate after magnitude pruning and retraining of the finetuned student at various sparsity levels.

1.1.2 `unsloth/Qwen2.5-7B-Instruct`

This has the same setup as the previous experiment, except the model used. Another change in this is pruning ΔW instead of the LoRA adapters. The reason for this with my intuition on how it could help is given in B. Another change is the training epochs of the student model. It is trained for 5 epochs now with the student finetuning of 3 epochs, with early stopping enabled if the trait preference doesn't increase in an epoch.

Note. An important thing we noticed was how subliminal learning evaluation was so fragile and highly dependent on specific evaluation prompts. This experiment used only one evaluation prompt. For this model, we noticed that the prompt caused it to always respond with pandas. Even while finetuning for owl preference trait, by the second epoch, the model had a 100% preference for pandas and 0% preference for owls (the owl preference was nearly 5% after the first epoch). This is the reason why we used panda preference for this experiment.

Implementation of Pruning adapter weights. One of the biggest challenges in running this experiment was finding a way to prune ΔW. The previous experiment pruned the LoRA adapters, which are much smaller than ΔW. Since all ΔW's concatenated makes it nearly 6.5B parameters (we need to concatenate because we do global pruning), pruning becomes really tough. CUDA has a limit of 2.1B in the size of vectors it can find the `kthvalue` of. Hence, we use a different technique. We do a single forward pass through the concatenated ΔW and get the highest magnitude weight. Then, we create nearly 10^6 bins, each containing a certain amount of values, between 0 and $ln(\Delta W_{max} + 1)$. We then assign the number of weights in each bin by taking the natural log of the absolute weights too, translated by 1. We then find the bin with the median value by counting the cumulative weights from bins, and that's how we get out cutoff pruning value. Then, we make a boolean mask, where any value below the cutoff is pruned. In this experiment, this happens in the CPU, which makes it really slow, and it takes nearly 10 minutes to prune.

Implementation of retraining. Initially, we tried to use a custom forward function with Unsloth, which multiplied ΔW with the pruning mask for each projection matrix. But, since Unsloth uses some optimized forward pass code, it skipped some projection matrices, in which pruning did not happen. To prevent this, we switched to using Transformers for finetuning the pruned student, but at the cost of a big increase in training time.

Results. Unfortunately, the results are unsatisfying after the effort we put into this experiment. For the first run, we get a 100% panda preference in the second epoch of student finetuning, and no owl preference, in the full-finetuned student and the pruned student. For the second run when we switched the student to evaluate for panda preference, we observe an almost 100% affinity for pandas in the full-finetuned student as well as the pruned students. Not attaching the graphs, since they are insignificant.

1.2 Magnitude Pruning without Re-training

One of the main reasons why we re-train the pruned model is to preserve capabilities. In this project our goal is to show that subnetworks are responsible for these behaviors and not to preserve capabilities. Hence, removing the re-training part from the previous section's setup seems promising. Another benefit this way is how the possibility of formation of refining of new subnetworks in the pruned network is removed. It also greatly reduces the training time and prevents the issues with re-training pruned networks we encountered earlier. In this section, we prune the LoRA adpaters and not ΔW for simplicity. Henceforth, we use `Qwen-2.5-7b-Instruct` for all experiments. We use the same pruning setup as in section 1.1.1. The evaluation prompts are randomly sampled from the set of 50 candidate prompts. We finetuned the student for 5 epochs, although that is essentially the same as 3 epochs, since we observed the trait transfer to peak at 3 epochs with nearly 17% and decline gradually after that.

Results. The results suggest that our hypothesis is true. At 50% sparsity, the pruned network retains half of the subliminal learning signal and the complement networks shows no signs of increase in trait preference even at the lowest sparsity level. It seems like this is the ideal setup for our experiment now. We can only focus on effective pruning techniques now.

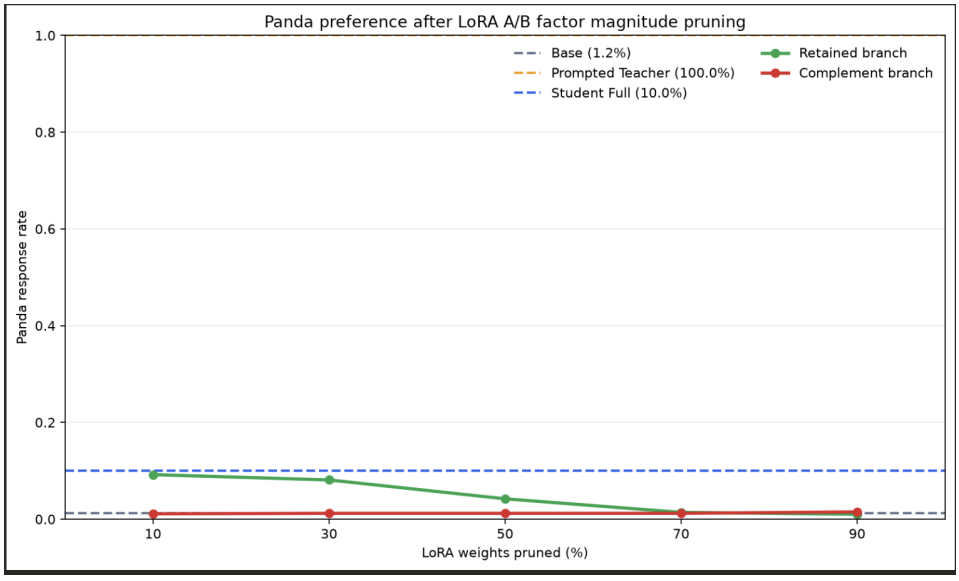


Figure 2: Owl preference rate after magnitude pruning without retraining of the finetuned student at various sparsity levels.

Analyzing pruning. To analyze the results further, we check what layers and projections contribute most to the pruned weights. The plot below represents the percentage of LoRA adapters of each projection matrix that were pruned. It shows a consistent pattern across layers, suggesting LoRA pruned layers equally. There is a preference of pruning certain types of projection matrices indicated in the results.

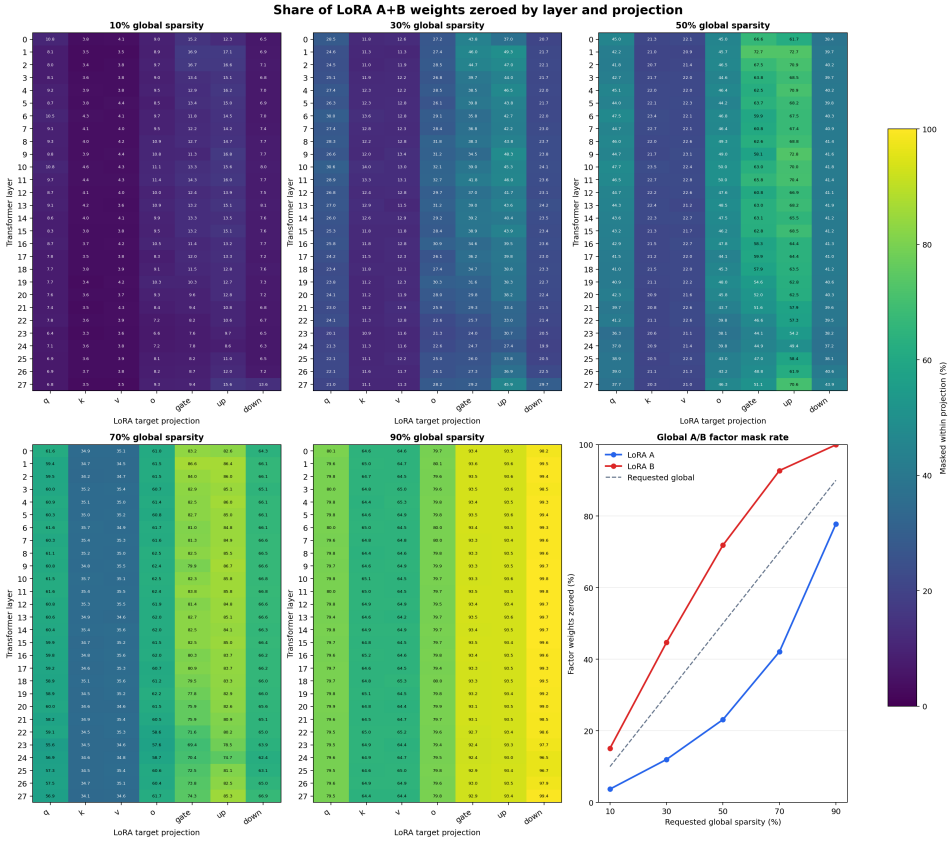


Figure 3: Analyzing contributions of each layer and projection matrix in pruning.

1.3 Logit Lens

The simplest improvement we can do on magnitude pruning is to identify certain layers in which the behavior is concentrated, and the easiest way to do that is logit lens. First of all, we try to use the usual convention of logit lens. Take the residual activations of each layer and pass it directly to the unembedding after the RMSNorm, skipping the rest of the layers.

Results. We don't observe any specific signal of a single layer contributing to trait expression.

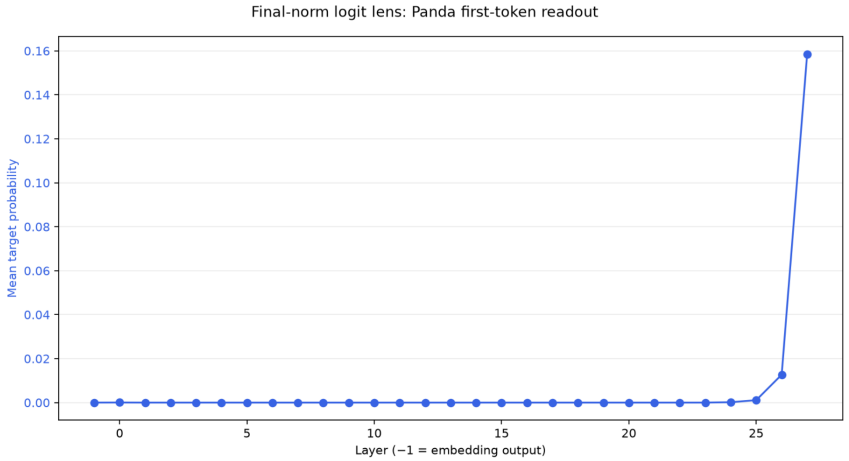


Figure 4: Logit lens on the whole model doesn't give any signal of trait concentration in layers.

Now we try a variation of logit lens. The core intuition behind this is that, since we use LoRA for finetuning and freeeze the base model weights, LoRA adapters will carry a richer signal of trait expression. Since LoRA weights lead to trait expression, we can try to just prune the LoRA adapters and leave the base model weights as it is. Rest of the setup remains the sane. The only change is that for each layer, instead of avoiding forward pass through the rest of the layers after it, we instead zero out the LoRA weights after that layer and just use the base model's weights.

Results. We observe clear concentration of the subliminal signal in the beginning layers of the model. Even with less than half the model's layers used, almost the whole trait transmission is captures. This motivates pruning only the early layers of the model. This suggests that the assumption of the highest norm weights corresponding to the trait assumption of magnitude pruning was violated in our case. The later layers have much lesser subliminal signal in comparison to the previous layers, but still receive equal pruning as the earlier layers. Even though the results are promising, one reason why this works is that the adapters in the earlier layers exploit the structure of the weights of the base model in the later layers, since pruning the weights of the base model also does not work.

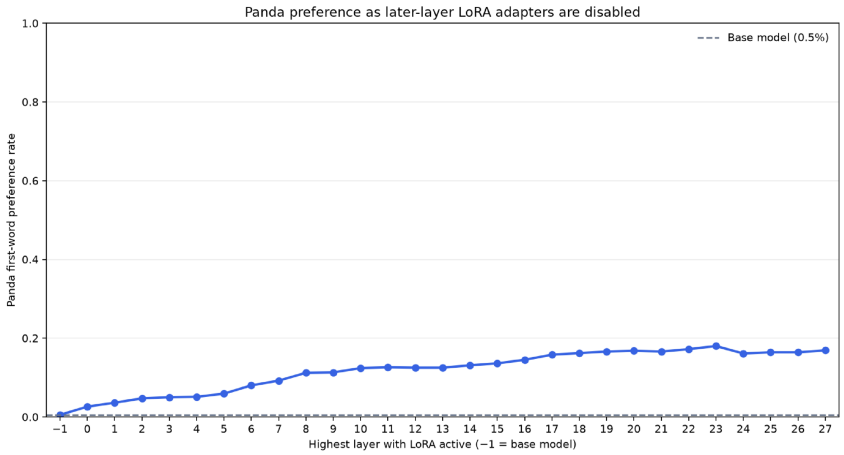


Figure 5: Logit lens on only LoRA adapters and keeping base model weights intact shows a signal of trait concentration in earlier layers.

1.4 Zero Ablation

Just to verify the results of logit lens, we do zero ablation on LoRA adapters of one layer. This would also give information on whether a specific early layer giving high contributions to trait transfer. Basically, in this experiment we take the finetuned student, and zero out LoRA adapters in each layer. Then we check for trait elicitation using the rest of the layer's adapters tuned on. We also do vice-versa. We choose zero ablation over other similar methods due to the ease of implementation and lesser running time.

Results. The results give a good validation for concentration of subliminal signal in the beginning layers, and indicate nearly equal important of all early layers.

1.5 Early Layers Pruning

Motivated by the previous two experiments, we now try to prune the early layers only and check for trait preference. Essentially, we follow the same setup as the logit lens on LoRA adapters setup, but along with that we prune the adapters of the earlier layers upto certrain sparsity levels. The core idea behind this setup is that, we exclude the later layers because they don't carry the subliminal signal, and only some weights of the earlier layers are part of the lottery ticket, so we can remove the rest.

Results. We see a considerable improvement from the previous magnitude pruning here. At layer 14, with 30% sparsity, there is almost full trait retention, with trait transfer being around 13-14%. This makes it a total of 65% sparsity, while in the magnitude pruning all layers experiment, trait transfer was nearly same as baseline at 70% sparsity. With these results, we now need to move on to methods that avoid the assumption that the biggest weights correspond to the subliminal signal.

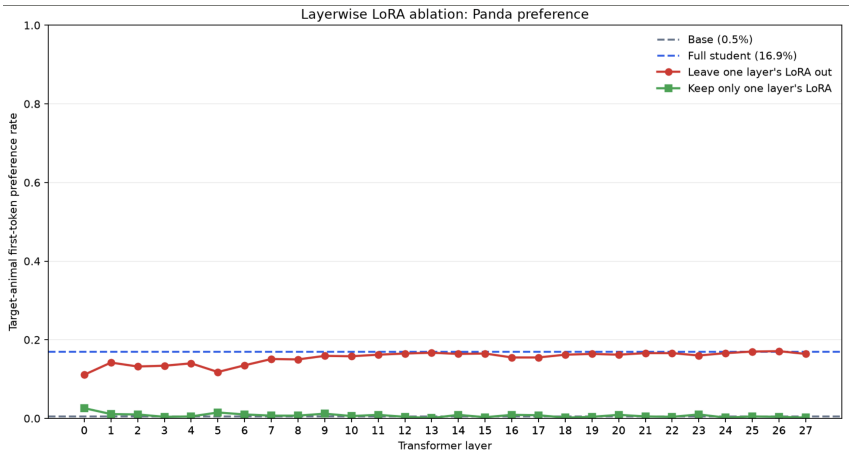


Figure 6: Zero Ablation of the full finetuned student to check for contribution of specific layers.

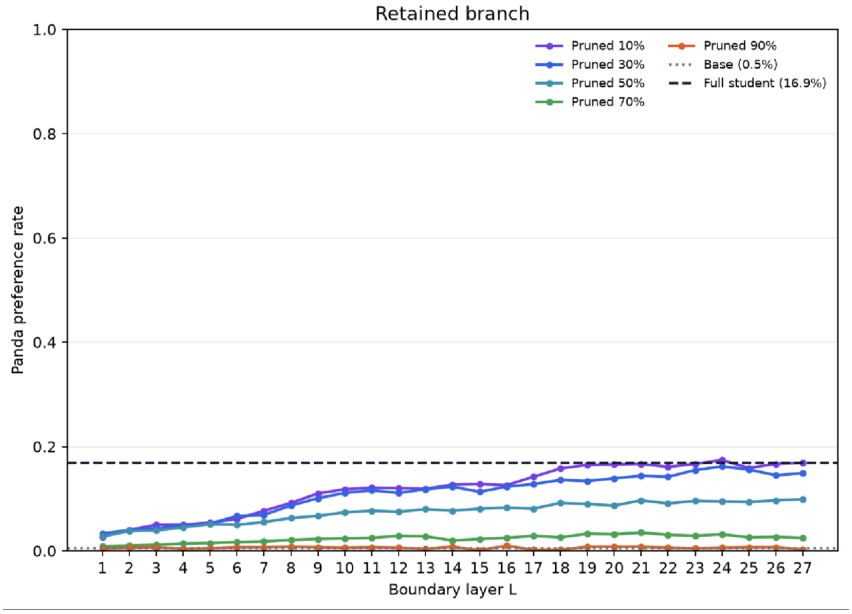


Figure 7: Magnitude Pruning to different sparsity levels after pruning all adapters of later layers.

1.6 Iterative Magnitude Pruning

I was thinking of using this, but after the results of magnitude pruning without retraining of the pruned student, it seems insignificant to do this. I feel like the main benefit of IMP is to restore the performance of the network after pruning, but here our goal isn't to uphold performance, it's to find out subnetworks which cause this behavior to occur. So it doesn't make sense to do this anymore.

1.7 Column Based Pruning

Since each column in a weight matrix corresponds a certain feature in the activation space, I feel like just pruning for featues could help in effective pruning, since we can think of subliminal learning as increase in the magnitude of the target trait feature (the weights corresponding to it) in the activation space. Not sure if running would be a good idea, but can be easy to implement by pruning the lora adapters responsible for that column only and no other column.

1.8 Steering Vectors Pruning

Experiment still being designed...

1.9 Adaptive Opimizers pruning

Need to check the exact specifics of the code in the literature, since SL has now been showed to occur with non adaptive optimizers too.

Other things: Investigate label smoothing, gradient clipping

A Hyperparameters

Coming soon...

B Pruning LoRA Adapters vs. LoRA Weights

The core bottleneck in directly pruning LoRA adapters is how one adapter value corresponds to multiple values in the complete weight matrix and how it is combined with other weight values (if rank is more than 1) to form a single weight value.

Full thing coming soon...